

1) What are the types of Applications?

A. Applications can be classified into:

1. Web Applications - Run on browsers (e.g., YouTube, Gmail).
2. Desktop Applications - Installed locally (e.g., MS Word).
3. Mobile Applications - Run on smartphones (e.g., WhatsApp).
4. Console Applications - Text-based, run in terminals.
5. Enterprise Applications - Large-scale for organizations (e.g., ERP).
6. Embedded Applications - Run on hardware devices (e.g., ATM software).

2) What is programming?

A. Programming is the process of writing instructions (code) that a computer can execute to perform specific tasks or solve problems.

3) What is Python?

A. Python is a high-level, interpreted, object-oriented programming language known for its simple syntax and large standard library.

It is widely used in web development, data science, AI, machine learning, and automation.

4) Python program to check if a number is positive, negative, or zero

A. `num = float(input("Enter a number: "))`

`if num > 0:`

`print("Positive number")`

`elif num < 0:`

`print("Negative number")`

`else:`

`print("Zero")`

Sample Output:

Enter a number: -5

Negative number

5) Write a Python program to get the Factorial number of given numbers.

```
A. num = int(input("Enter a number: "))
factorial = 1
if num < 0:
    print("Factorial does not exist for negative numbers")
elif num == 0:
    print("Factorial is 1")
else:
    for i in range(1, num + 1):
        factorial *= i
    print("Factorial of", num, "is", factorial)
```

Sample Output:

```
Enter a number: 5
Factorial of 5 is 120
```

6) Write a Python program to get the Fibonacci series of given range.

```
A. n = int(input("Enter the range: "))
a, b = 0, 1
print("Fibonacci series:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Sample Output:

```
Enter the range: 7
Fibonacci series:
0 1 1 2 3 5
```

7) How memory is managed in Python?

A. Python memory is managed through:

- **Private heap space:** Stores all Python objects and data structures.
- **Memory Manager:** Allocates heap space for Python objects.
- **Garbage Collector:** Automatically deallocates memory for unused objects using reference counting and cyclic garbage collection.
- **Built-in functions:** Like `id()`, `type()`, `sys.getsizeof()` help inspect memory usage.

8) What is the purpose continuing statement in python?

A. The `continue` statement skips the rest of the code inside the current loop iteration and jumps to the next iteration of the loop.

Example:

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output: 0, 1, 3, 4

9) Write python program that swap two number with temp variable and without temp variable.

A. With temp variable:

```
a = 5,b = 10
temp = a
a = b
b = temp
print("a =", a, "b =", b)
```

Without temp variable:

```
a = 5,b = 10
a, b = b, a
print("a =", a, "b =", b)
```

10) Write a Python program to find whether a given number is even or odd, print out an appropriate message to the user.

```
A.num = int(input("Enter a number: "))

if num % 2 == 0:

    print("Even number")

else:
    print("Odd number")
```

11) Write a Python program to test whether a passed letter is a vowel or not.

```
A. letter = input("Enter a letter: ").lower()
```

```
if letter in 'aeiou':  
    print("It is a vowel.")  
else:  
    print("It is not a vowel.")
```

12) Write a Python program to sum of three given integers. However, if two values are equal sum will be zero.

```
A. def custom_sum(a, b, c):  
  
    if a == b or b == c or a == c:  
        return 0  
    return a + b + c
```

```
print(custom_sum(1, 2, 3)) # 6  
print(custom_sum(2, 2, 3)) # 0
```

13) Write a Python program that will return true if the two given integer values are equal or their sum or difference is 5.

```
A. def check(a, b):  
  
    return a == b or abs(a - b) == 5 or (a + b) == 5
```

```
print(check(2, 3)) # True (sum is 5)  
print(check(7, 2)) # True (difference is 5)
```

14) Write a python program to sum of the first n positive integers.

```
A.n = int(input("Enter a number: "))  
  
sum_n = n * (n + 1) // 2  
  
print("Sum is:", sum_n)
```

15) Write a Python program to calculate the length of a string.

```
A.string = input("Enter a string: ")  
  
print("Length is:", len(string))
```

16) Write a Python program to count the number of characters (character frequency) in a string.

```
A.string = input("Enter a string: ")  
  
freq = {}  
  
for char in string:  
    freq[char] = freq.get(char, 0) + 1  
print(freq)string = input("Enter a string: ")  
freq = {}  
for char in string:  
    freq[char] = freq.get(char, 0) + 1  
print(freq)
```

17) What are negative indexes and why are they used?

A. Negative indexes allow access to elements from the **end** of a list/string.

- **-1** refers to the last element, **-2** the second-last, etc.

Example:

```
s = "Python"

print(s[-1])  # Output: 'n'
```

18) Write a Python program to count occurrences of a substring in a string.

A. `text = "Python is fun. Python is easy."`

```
count = text.count("Python")
print("Occurrences of 'Python':", count)
```

19) Write a Python program to count the occurrences of each word in a Page 17 of 28 given sentence

A. `sentence = input("Enter a sentence: ")`

```
words = sentence.split()
```

```
freq = {}
```

```
for word in words:
```

```
    freq[word] = freq.get(word, 0) + 1
```

```
print(freq)
```

20) Write a Python program to get a single string from two given strings, separated by a space and swap the first two characters of each string.

```
A. def mix_strings(a, b):
```

```
    new_a = b[:2] + a[2:]
```

```
    new_b = a[:2] + b[2:]
```

```
    return new_a + ' ' + new_b
```

```
print(mix_strings("abc", "xyz")) # Output: "xyc abz"
```

21) Write a Python program to add 'in' at the end of a given string (length should be at least 3). If the given string already ends with 'ing' then add 'ly' instead if the string length of the given string is less than 3, leave it unchanged.

```
string = input("Enter a string: ")
```

```
if len(string) < 3:
```

```
    print(string)
```

```
elif string.endswith("ing"):
```

```
    print(string + "ly")
```

```
else:
```

```
    print(string + "ing")
```

Sample Output:

Enter a string: play

Playing

22) Write a Python function to reverses a string if its length is a multiple of 4.

```
def reverse_if_multiple_of_4(s):  
    if len(s) % 4 == 0:  
        return s[::-1]  
    return s  
  
print(reverse_if_multiple_of_4("abcd"))
```

Sample Output:

dcba

23) Write a Python program to get a string made of the first 2 and the last 2 chars from a given a string. If the string length is less than 2, return instead of the empty string.

```
string = input("Enter a string: ")  
  
if len(string) < 2:  
    print("")  
  
else:  
    print(string[:2] + string[-2:])
```

Sample Output:

Enter a string: Python

Pyon

24) Write a Python function to insert a string in the middle of a string.

```
def insert_middle(s1, s2):  
    mid = len(s1) // 2  
    return s1[:mid] + s2 + s1[mid:]  
  
print(insert_middle("Python", "ABC"))
```

Sample Output:

PyABCthon

25) What is List? How will you reverse a list?

List: A collection of ordered, mutable items in Python.

Reverse a list:

```
my_list = [1, 2, 3]  
  
my_list.reverse()  
  
print(my_list)  
  
OR  
  
print(my_list[::-1])
```

26) How will you remove last object from a list?

```
my_list = [1, 2, 3]
```

```
my_list.pop()
```

```
print(my_list)
```

Sample Output:

```
[1, 2]
```

27) Suppose list1 is [2, 33, 222, 14, and 25], what is list1 [1]?

Answer: 33

28) Differentiate between append () and extend () methods?

append(x) → Adds a single element at the end.

extend(iterable) → Adds multiple elements from another iterable.

Example:

```
l = [1, 2]
```

```
l.append([3, 4]) # [1, 2, [3, 4]]
```

```
l.extend([3, 4]) # [1, 2, 3, 4]
```

29) Write a Python function to get the largest number, smallest num and sum of all from a list.

```
nums = [10, 20, 5, 7]

print("Largest:", max(nums))

print("Smallest:", min(nums))

print("Sum:", sum(nums))
```

Sample Output:

Largest: 20

Smallest: 5

Sum: 42

30) How will you compare two lists?

```
list1 = [1, 2, 3]

list2 = [1, 2, 3]

print(list1 == list2) # True if same elements in same order
```

31) Write a Python program to count the number of strings where the string length is 2 or more and the first and last character are same from a given list of strings.

```
words = ["abc", "xyz", "aba", "1221"]
```

```
count = 0
```

```
for w in words:
```

```
    if len(w) >= 2 and w[0] == w[-1]:
```

```
        count += 1
```

```
print(count)
```

Sample Output:

2

32) Write a Python program to remove duplicates from a list.

```
my_list = [1, 2, 2, 3, 1]
```

```
unique_list = list(set(my_list))
```

```
print(unique_list)
```

Sample Output:

[1, 2, 3]

33) Write a Python program to check a list is empty or not.

```
my_list = []  
  
if not my_list:  
  
    print("List is empty")
```

Sample Output:

List is empty

34) Write a Python function that takes two lists and returns true if they have at least one common member.

```
def common_member(l1, l2):  
  
    return any(i in l2 for i in l1)  
  
print(common_member([1, 2, 3], [4, 5, 2]))
```

Sample Output:

True

35) Write a Python program to generate and print a list of first and last 5 elements where the values are square of numbers between 1 and 30.

```
squares = [i**2 for i in range(1, 31)]  
  
print(squares[:5])  
  
print(squares[-5:])
```

Sample Output:

[1, 4, 9, 16, 25]

[676, 729, 784, 841, 900]

36) Write a Python function that takes a list and returns a new list with unique elements of the first list.

```
def unique_elements(lst):  
    return list(set(lst))  
  
print(unique_elements([1, 2, 2, 3]))
```

Sample Output:

[1, 2, 3]

37) Write a Python program to convert a list of characters into a string.

```
chars = ['h', 'e', 'l', 'l', 'o']  
  
string = ''.join(chars)  
  
print(string)
```

Sample Output:

hello

38) Write a Python program to select an item randomly from a list.

```
import random

my_list = [10, 20, 30, 40]

print(random.choice(my_list))
```

Sample Output:

30

39) Write a Python program to find the second smallest number in a list.

```
nums = [1, 5, 2, 8]

nums.sort()

print(nums[1])
```

Sample Output:

2

40) Write a Python program to get unique values from a list

```
my_list = [1, 2, 2, 3, 4, 4]

unique_values = list(set(my_list))

print(unique_values)
```

Sample Output:

[1, 2, 3, 4]

41) Write a Python program to check whether a list contains a sub list.

```
def is_sublist(lst, sublist):  
    for i in range(len(lst) - len(sublist) + 1):  
        if lst[i:i+len(sublist)] == sublist:  
            return True  
    return False  
  
print(is_sublist([1, 2, 3, 4, 5], [2, 3]))
```

Sample Output:

True

42) Write a Python program to split a list into different variables.

```
my_list = [1, 2, 3]  
  
a, b, c = my_list  
  
print(a, b, c)
```

Sample Output:

1 2 3

43) What is tuple? Difference between list and tuple.

Tuple: Immutable, ordered collection of elements.

List vs Tuple:

Lists are mutable, tuples are immutable. Lists use [], tuples use ().

44) Write a Python program to create a tuple with different data types.

```
my_tuple = (1, "Hello", 3.14, True)
```

```
print(my_tuple)
```

Sample Output:

```
(1, 'Hello', 3.14, True)
```

45) Write a Python program to unzip a list of tuples into individual lists. Page 19 of 28.

```
list_of_tuples = [(1, 'a'), (2, 'b'), (3, 'c')]
```

```
numbers, letters = zip(*list_of_tuples)
```

```
print(list(numbers))
```

```
print(list(letters))
```

Sample Output:

```
[1, 2, 3]
```

```
['a', 'b', 'c']
```

46) Write a Python program to convert a list of tuples into a dictionary.

```
list_of_tuples = [('a', 1), ('b', 2)]
```

```
dictionary = dict(list_of_tuples)
```

```
print(dictionary)
```

Sample Output: {'a': 1, 'b': 2}

47) How will you create a dictionary using tuples in python?

```
tuple_data = (('x', 1), ('y', 2))
```

```
dictionary = dict(tuple_data)
```

```
print(dictionary)
```

Sample Output: {'x': 1, 'y': 2}

48) Write a Python script to sort (ascending and descending) a dictionary by value.

```
my_dict = {'a': 3, 'b': 1, 'c': 2}
```

```
asc = dict(sorted(my_dict.items(), key=lambda x: x[1]))
```

```
desc = dict(sorted(my_dict.items(), key=lambda x: x[1], reverse=True))
```

```
print("Ascending:", asc)
```

```
print("Descending:", desc)
```

Sample Output:

Ascending: {'b': 1, 'c': 2, 'a': 3}

Descending: {'a': 3, 'c': 2, 'b': 1}

49) Write a Python script to concatenate following dictionaries to create a new one.

```
dict1 = {'a': 1, 'b': 2}
```

```
dict2 = {'c': 3}
```

```
dict3 = **dict1, **dict2
```

```
print(dict3)
```

Sample Output:

```
{'a': 1, 'b': 2, 'c': 3}
```

50) Write a Python script to check if a given key already exists in a dictionary.

```
my_dict = {'a': 1, 'b': 2}
```

```
print('a' in my_dict)
```

Sample Output:

```
True
```

51) How Do You Traverse Through a Dictionary Object in Python?

```
my_dict = {'a': 1, 'b': 2}
```

```
for key, value in my_dict.items():
```

```
    print(key, value)
```

Sample Output: a 1 b 2

52) How Do You Check the Presence of a Key in A Dictionary?

```
my_dict = {'a': 1, 'b': 2}
```

```
if 'b' in my_dict:
```

```
    print("Key exists")
```

Sample Output:

Key exists

53) Write a Python script to print a dictionary where the keys are numbers between 1 and 15.

```
my_dict = {x: x**2 for x in range(1, 16)}
```

```
print(my_dict)
```

Sample Output:{1: 1, 2: 4, ..., 15: 225}

54) Write a Python program to check multiple keys exists in a dictionary.

```
my_dict = {'a': 1, 'b': 2, 'c': 3}
```

```
keys = ['a', 'c']
```

```
print(all(k in my_dict for k in keys))
```

Sample Output:True

55) Write a Python script to merge two Python dictionaries.

```
dict1 = {'a': 1, 'b': 2}
```

```
dict2 = {'c': 3}
```

```
dict1.update(dict2)
```

```
print(dict1)
```

Sample Output: {'a': 1, 'b': 2, 'c': 3}

56) Write a Python program to map two lists into a dictionary Sample output: Counter ({'a': 400, 'b': 400,'d': 400, 'c': 300}). Page 20 of 28.

```
keys = ['a', 'b', 'c', 'd']
```

```
values = [400, 400, 300, 400]
```

```
my_dict = dict(zip(keys, values))
```

```
print(my_dict)
```

Sample Output: {'a': 400, 'b': 400, 'c': 300, 'd': 400}

57) Write a Python program to find the highest 3 values in a dictionary.

```
my_dict = {'a': 10, 'b': 50, 'c': 40, 'd': 20}
```

```
top3 = sorted(my_dict.values(), reverse=True)[:3]
```

```
print(top3)
```

Sample Output: [50, 40, 20]

58) Write a Python program to combine values in python list of dictionaries.
Sample data: [{ 'item': 'item1', 'amount': 400}, { 'item': 'item2', 'amount': 300}, o { 'item': 'item1', 'amount': 750}] Expected Output: • Counter ({ 'item1': 1150, 'item2': 300}).

```
from collections import Counter

data = [{ 'item': 'item1', 'amount': 400}, { 'item': 'item2', 'amount': 300},
{ 'item': 'item1', 'amount': 750}]

counter = Counter()

for d in data:

    counter[d['item']] += d['amount']

print(counter)
```

Sample Output: Counter({ 'item1': 1150, 'item2': 300})

59) Write a Python program to create a dictionary from a string. Note:
Track the count of the letters from the string.

```
string = "w3resource"

letter_count = {}

for char in string:

    letter_count[char] = letter_count.get(char, 0) + 1

print(letter_count)
```

Sample Output: { 'w': 1, '3': 1, 'r': 2, 'e': 2, 's': 1, 'o': 1, 'u': 1, 'c': 1}

60) Sample string: 'w3resource' Expected output: • {'3': 1, 's': 1, 'r': 2, 'u': 1, 'w': 1, 'c': 1, 'e': 2, 'o': 1}

```
string = "w3resource"
```

```
print({char: string.count(char) for char in string})
```

Sample Output: {'w': 1, '3': 1, 'r': 2, 'e': 2, 's': 1, 'o': 1, 'u': 1, 'c': 1}

61) Write a Python function to calculate the factorial of a number (a nonnegative integer).

```
def factorial(n):
```

```
    if n < 0:
```

```
        return "Invalid input"
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    return n * factorial(n - 1)
```

62) Write a Python function to check whether a number is in a given range

```
def in_range(num, start, end):
```

```
    return start <= num <= end
```

63) Write a Python function to check whether a number is perfect or not.

```
def is_perfect(n):
```

```
    if n <= 0:        return False
```

```
    return sum(i for i in range(1, n) if n % i == 0) == n
```

64) Write a Python function that checks whether a passed string is palindrome or not.

```
def is_palindrome(s):
```

```
    s = s.lower().replace(" ", "") # Optional: clean up input
```

```
    return s == s[::-1]
```

65) How Many Basic Types of Functions Are Available in Python? 66) How can you pick a random item from a list or tuple?

Python has **two basic types** of functions:

1. **Built-in functions** - Predefined (e.g., `len()`, `print()`, `max()`)
2. **User-defined functions** - Created by users using `def`

67) How can you pick a random item from a range?

```
import random
```

```
random.choice(['apple', 'banana', 'cherry'])
```

68) How can you get a random number in python?

```
import random
```

```
random.randint(1, 100) # Random integer between 1 and 100
```

69) How will you set the starting value in generating random numbers? Page 21 of 28.

```
import random
```

```
random.seed(10)
```

70) How will you randomize the items of a list in place?

```
import random
```

```
my_list = [1, 2, 3, 4, 5]
```

```
random.shuffle(my_list)
```

71) What is File function in python? What are keywords to create and write file.

File function in Python is used to **perform file operations** such as open, read, write, and close a file.

The **open()** function is used to open a file.

File modes (keywords):

'r' - Read (default)

'w' - Write (creates file if it doesn't exist; overwrites if it does)

'a' - Append

'x' - Create a new file

'b' - Binary mode

't' - Text mode (default)

72) Write a Python program to read an entire text file.

with open('sample.txt', 'r') as file:

```
    content = file.read()
```

```
    print(content)
```

73) Write a Python program to append text to a file and display the text.

Append text

with open('sample.txt', 'a') as file:

```
    file.write("\nThis is appended text.")
```

Display updated content

with open('sample.txt', 'r') as file:

```
    print(file.read())
```

74) Write a Python program to read first n lines of a file.

```
def read_first_n_lines(filename, n):
```

```
    with open(filename, 'r') as file:
```

```
        for i in range(n):
```

```
            print(file.readline(), end='')
```

```
read_first_n_lines('sample.txt', 3)
```

75) Write a Python program to read last n lines of a file.

```
def read_last_n_lines(filename, n):
```

```
    with open(filename, 'r') as file:
```

```
        lines = file.readlines()
```

```
        for line in lines[-n:]:
```

```
            print(line, end='')
```

```
read_last_n_lines('sample.txt', 3)
```

76) Write a Python program to read a file line by line and store it into a list

with open('sample.txt', 'r') as file:

```
    lines_list = file.readlines()
```

```
print(lines_list)
```

77) Write a Python program to read a file line by line store it into a variable.

with open('sample.txt', 'r') as file:

```
    lines = ""
```

```
    for line in file:
```

```
        lines += line
```

```
print(lines)
```

78) Write a python program to find the longest words.

```
def find_longest_words(filename):
```

```
    with open(filename, 'r') as file:
```

```
        words = file.read().split()
```

```
        longest = max(words, key=len)
```

```
        print("Longest word:", longest)
```

```
find_longest_words('sample.txt')
```

79) Write a Python program to count the number of lines in a text file.

with open('sample.txt', 'r') as file:

 line_count = sum(1 for line in file)

print("Number of lines:", line_count)

80) Write a Python program to count the frequency of words in a file.

from collections import Counter

A. with open('sample.txt', 'r') as file:

 words = file.read().lower().split()

 word_count = Counter(words)

print(word_count)

81) Write a Python program to write a list to a file.

data = ["apple", "banana", "cherry"]

with open("fruits.txt", "w") as file:

 for item in data:

 file.write(item + "\n")

82) Write a Python program to copy the contents of a file to another file.

```
A. with open("source.txt", "r") as src, open("destination.txt", "w") as dest:  
    dest.write(src.read())
```

83) Explain Exception handling? What is an Error in Python?

Exception handling in Python is a way to manage runtime errors, so the program doesn't crash.

Errors are issues in a program that can be:

Syntax Errors (e.g., missing colon, unmatched brackets)

Exceptions (e.g., ZeroDivisionError, FileNotFoundError)

We use try, except, else, and finally blocks to handle exceptions.

84) How many except statements can a try-except block have? Name Some built-in exception classes:

```
try: x = 1 / 0
```

```
except ZeroDivisionError:
```

```
    print("Division by zero!")
```

```
except ValueError:
```

```
    print("Invalid value")
```

some built-in exception classes:

ZeroDivisionError

ValueError

TypeError
IndexError
FileNotFoundError
KeyError

85) When will the else part of try-except-else be executed?

```
try: x = 10 / 2

except ZeroDivisionError:

    print("Error!")

else:

    print("No errors, result:", x)
```

86) Can one block of except statements handle multiple exception?

```
try:

    # some code

    pass

except (ValueError, TypeError):

    print("Caught a ValueError or TypeError")
```

87) When is the finally block executed?

```
Try:    x = 5 / 1
```

```
except ZeroDivisionError:
```

```
    print("Error!")
```

```
finally:
```

```
    print("This will always run")
```

88) What happens when „1“== 1 is executed?

```
print('1' == 1) # Output: False
```

Python checks the **types and values**.

'1' is a **string**, 1 is an **integer** → So, the result is False.

89) How Do You Handle Exceptions with Try/Except/Finally in Python?
Explain with coding snippets.

```
try:
```

```
    number = int(input("Enter a number: "))
```

```
    result = 10 / number
```

```
except ZeroDivisionError:
```

```
    print("You can't divide by zero!")
```

```
except ValueError:
```

```
    print("Please enter a valid integer.")
```

else:

```
    print("Result is:", result)
```

finally:

```
    print("Execution completed.")
```

90) Write python program that user to enter only odd numbers, else will raise an exception.

```
def get_odd_number():
```

```
    try:
```

```
        num = int(input("Enter an odd number: "))
```

```
        if num % 2 == 0:
```

```
            raise ValueError("This is not an odd number!")
```

```
        print("You entered an odd number:", num)
```

```
    except ValueError as ve:
```

```
        print("Error:", ve)
```

```
get_odd_number()
```